



ADDIS ABABA
**SCIENCE AND
TECHNOLOGY**
UNIVERSITY
UNIVERSITY FOR INDUSTRY

College of Engineering

Department of Software Engineering

Database Systems [SWEG2108]

Project title: Report on Design and Implementation of a Hybrid Logistics Management System
Section-B (Group-4)

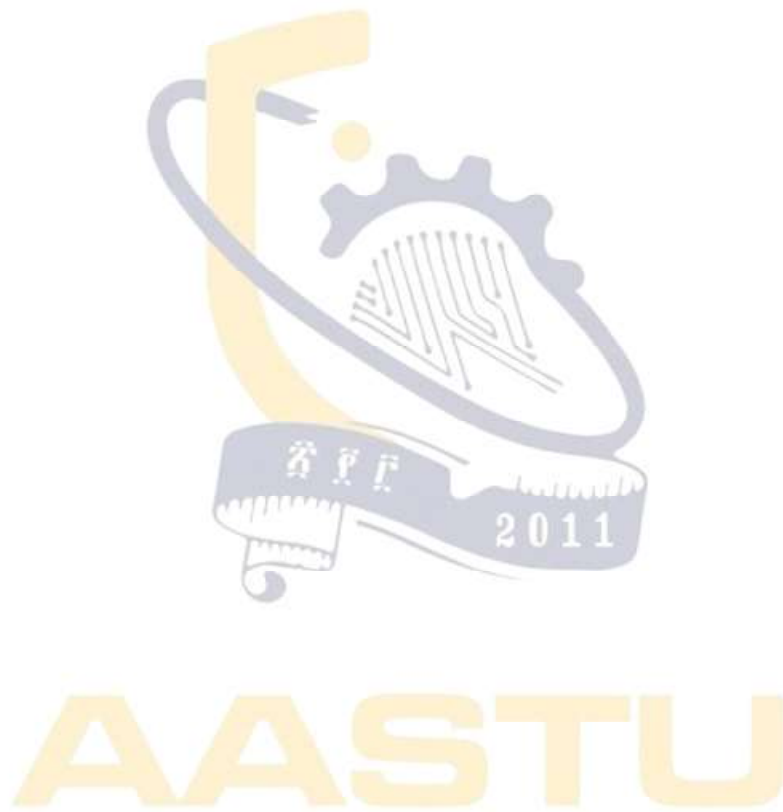
Name	ID No
1. Edom Anteneh	ETS 0466/17
2. Efrata Assefa	ETS 0468/17
3. Eleham Hassen	ETS 0473/17
4. Emnet Belachew	ETS 0492/17
5. Fikirte Alemayehu	ETS 0590/17
6. Finot Solomon	ETS 0599/17

Submitted To: Mr. Yaynshet Medhim

Submission Date: May , 2026

Acknowledgment

We would like to thank our instructor, Mr. Yaynshet Medhin, for his insightful lectures throughout this course. He helped us deepen our understanding of Modular Programming, Database Management Systems, and the intricacies of both Relational (SQL) and Non-Relational (NoSQL) architectures. This project serves as a comprehensive framework that will showcase our comprehension of those core concepts towards the implementation through the logistics-focused program we plan to build.



Introduction

The global logistics industry demands high-precision data management to ensure the seamless flow of goods, a challenge addressed in this project through the design and implementation of a robust Logistics Management System. Utilizing a hybrid database architecture, the system leverages MySQL to maintain strict Boyce-Codd Normal Form (BCNF) for transactional integrity across eight core tables, while incorporating MongoDB to provide the flexible, high-speed retrieval necessary for customer activity and order histories. Developed under an Agile methodology, the project evolved through iterative cycles of normalization and testing to eliminate data anomalies and optimize performance. Ultimately, this implementation provides a scalable and reliable foundation for generating critical business insights—ranging from inventory stock alerts to courier efficiency reports—that are essential for modern, data-driven supply chain decision-making.

Chapter 1: Analysis & Documentation

1.1 Background of the Organization

The theoretical organization under analysis is a premier **Logistics Company** and supply chain management provider operating within East Africa, with its primary headquarters and central hub located in Addis Ababa, Ethiopia. Established to bridge the gap between emerging local manufacturing sectors, international import gateways, and a rapidly expanding consumer market, the company specializes in comprehensive warehousing, regional distribution, and critical "last-mile" urban delivery.

In the landscape of modern commerce, logistics providers serve as the physical backbone of the economy. For this **Logistics Management System**, operations span a complex network of physical warehouses located in major industrial zones such as Bole Lemi, Hawassa, and Adama. These facilities receive bulk shipments from domestic producers and international shipping lanes via the Djibouti-Addis Ababa transit corridor. Once received, goods must be cataloged, safely stored, and dynamically routed to retail outlets, regional distribution nodes, and individual domestic consumers.

The logistical landscape of Ethiopia presents unique geographical, infrastructural, and operational opportunities. Urban centers are experiencing exponential growth, driving an unprecedented demand for reliable delivery services. However, the physical movement of goods is highly dependent on a variety of unpredictable factors, including road conditions, fluctuating vehicle availability, and varying regional administrative requirements. To maintain a competitive edge, minimize transport overhead, and guarantee delivery timelines,

the **Logistics Company** relies on a complex operational matrix. This matrix requires the seamless coordination of physical assets, human resources, and real-time operational intelligence.

To support these complex physical activities, the organization's administrative structure is divided into three distinct operational divisions:

- **The Warehouse Management Division:** Responsible for space allocation, inventory control, stock replenishment, and order picking across all regional hubs.
- **The Fleet and Dispatch Division:** Manages physical transport assets, coordinates with internal and contract couriers, optimizes transit routes, and schedules daily delivery dispatches.
- **The Customer Relations and Order Fulfillment Division:** Handles incoming client requests, manages customer service profiles, processes financial invoices, and monitors service level agreement (SLA) compliance.

Traditionally, these divisions operated in functional silos, relying on localized spreadsheets, paper ledger systems, and independent, disconnected software applications. As the volume of daily transactions grew from hundreds to tens of thousands of parcels, this fragmented administrative approach became untenable. The lack of a unified, enterprise-wide database system resulted in widespread operational inefficiencies.

The implementation of the **Logistics Management System** represents a strategic, technology-driven transformation. By replacing legacy systems with a unified database architecture, the organization aims to centralize its operational data, streamline cross-departmental workflows, and establish a single, verifiable source of truth for every transaction, product, and shipment within its network.

1.2 Problem Statement

Despite its strong market position and extensive physical network, the **Logistics Company** has been severely constrained by the limitations of its legacy data management infrastructure. The absence of a centralized, scalable, and integrated database system has created critical operational bottlenecks that threaten the viability of the business. These core issues can be categorized into three main areas:

1.2.1 Data Fragmentation and Systemic Inconsistency

Because individual departments developed their own localized tracking methods, critical data was highly fragmented across different formats and storage media. The Warehouse Management Division tracked stock levels using localized spreadsheets, while the Fleet and

Dispatch Division relied on manual trip manifests and independent messaging apps to coordinate with couriers.

This separation created massive data discrepancies. For example, a product marked as "In Stock" by a warehouse manager might already have been dispatched by a fleet controller, leading to duplicate bookings and system-wide confusion. Without a centralized database to enforce referential integrity and data validation, human errors like duplicate entries, misspelled customer addresses, and mismatched product SKU codes became common. These errors corrupted the integrity of operational reports and undermined management's decision-making capabilities.

1.2.2 Lack of Real-Time Visibility and Shipment Tracking

In the modern logistics industry, real-time visibility is no longer a luxury—it is a baseline customer expectation. Legacy systems were entirely batch-oriented; dispatch records were manually updated only at the end of each business day. Consequently, customer support teams could not provide immediate updates on package locations or estimated delivery times. If a customer inquired about a critical shipment, staff had to manually call dispatchers, who then phoned individual couriers, creating a slow and frustrating communication loop. This lack of transparency led to high customer churn, increased call volumes to support centers, and an inability to offer competitive SLAs to corporate clients who require precise, minute-by-minute tracking.

1.2.3 Technical Scalability Constraints of Relational Models

As the organization attempted to modernize its operations by introducing automated tracking, the limitations of traditional relational database models became clear. Relational systems (like MySQL) are highly effective at handling structured transactions—such as processing customer orders, managing inventory, and maintaining financial records—thanks to their strict adherence to ACID (Atomicity, Consistency, Isolation, Durability) properties.

However, shipment tracking generates a continuous stream of high-velocity, semi-structured data, including frequent status changes, GPS coordinates, and delivery notes. Forcing this high-frequency stream into rigid, deeply joined relational tables caused severe database lockups, slow query performance, and system downtime during peak operational hours. The rigid schemas made it highly difficult to adapt to new types of tracking data, such as sensor readings or photo confirmations, without performing risky and disruptive database alterations.

1.3 Objectives

To resolve these operational and technical challenges, this project was established to design, develop, and implement an enterprise-grade database solution tailored specifically to the needs of the **Logistics Management System**.

1.3.1 General Objective

The primary objective of this project is to design and implement a robust, high-performance **Logistics Management System** utilizing a dual-engine database architecture (MySQL and MongoDB Atlas). This hybrid system is engineered to secure core transactional business data through strict relational constraints while enabling highly scalable, real-time shipment tracking via a flexible, document-oriented NoSQL database.

1.3.2 Specific Objectives

- **Design a Normalized Conceptual Model:** Analyze the operational workflows of the **Logistics Company** to identify all core entities, attributes, and business rules, translating them into a highly detailed Entity-Relationship Diagram (ERD).
- **Establish a Relational Database Schema:** Implement a fully normalized MySQL database structure up to Boyce-Codd Normal Form (BCNF) to manage structured transactional data, including customer profiles, orders, product catalogs, and warehouse inventories, while eliminating data redundancy.
- **Deploy a Scalable NoSQL Document Store:** Configure a cloud-based MongoDB Atlas cluster to house dynamic, high-velocity shipment tracking records and customer interaction logs, utilizing flexible document schemas that allow for seamless data updates.
- **Implement Cross-Platform Data Synchronization:** Design and implement a standardized data-mapping strategy that aligns unique identifiers (such as `Customer_ID` and `Order_ID`) as consistent INT types across both relational tables and NoSQL documents, establishing seamless data integration.
- **Optimize Database Query Performance:** Write, test, and optimize complex SQL queries, views, and MongoDB aggregation pipelines to generate real-time operational reports, such as inventory levels, courier delivery speeds, and regional shipment volumes.
- **Establish Professional Version Control Standards:** Manage the development lifecycle of the database scripts through a collaborative GitHub repository, adhering to a multiple commits workflow to simulate industry-standard code deployment and peer review processes.

1.4 Scope and Benefits

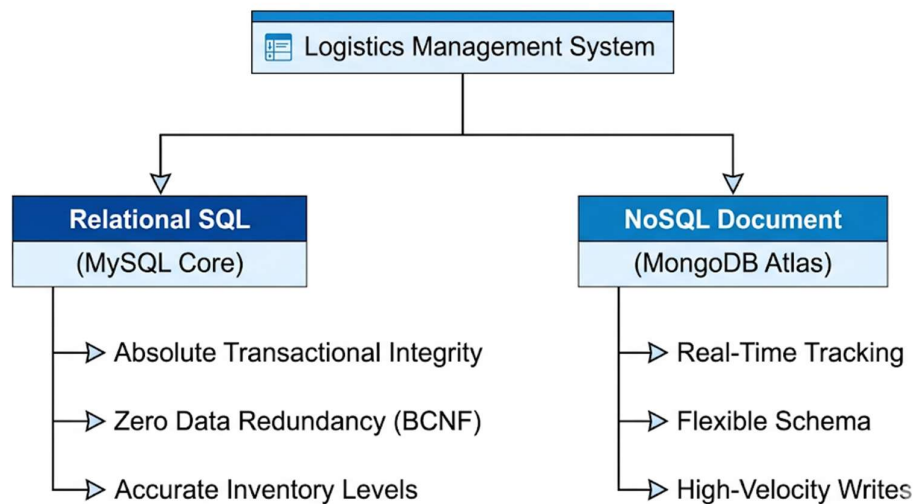
1.4.1 Scope of the System

The scope of the **Logistics Management System** is strategically defined to manage the core operational lifecycle of third-party logistics. It encompasses the following functional domains:

- **Customer Relationship Management (CRM) Data Layer:** Storing and managing comprehensive customer profiles, physical delivery addresses, contact information, and account statuses.
- **Warehouse and Inventory Control:** Tracking physical warehouse locations, total storage capacities, and real-time inventory balances across multiple regional hubs, including automated calculations of quantities on hand.
- **Order Processing and Transaction Management:** Recording customer purchase transactions, itemizing order details, tracking payment statuses, and initiating the fulfilment pipeline.
- **Fulfilment and Courier Logistics:** Managing courier profiles, transport vehicle specifications, driver licensing, active delivery assignments, and shipment routing.
- **Dynamic Shipment Tracking:** Storing real-time delivery status updates, tracking history logs, transit timestamps, and final delivery confirmations.

The system is designed as a foundational data layer. While it establishes the underlying database schemas, indexing strategies, and relational constraints, it does not include the creation of end-user graphical interfaces (GUIs), real-time GPS hardware integration, or automated payment processing gateways. These components are reserved for future application-layer integrations that will connect directly to this database foundation.

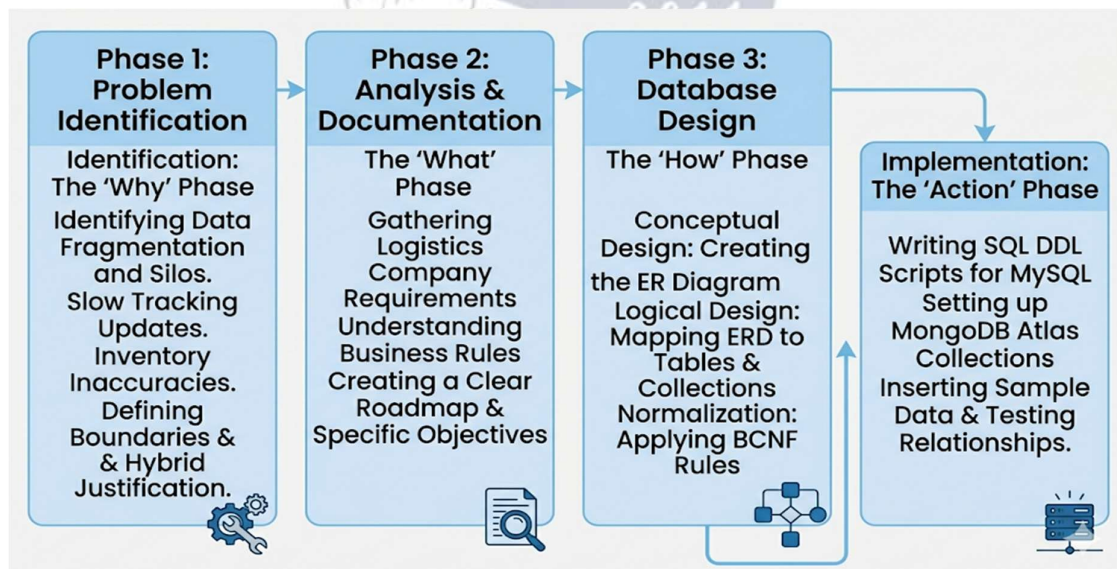
1.4.2 Benefits to the Organization



Implementing this hybrid database architecture brings significant operational and strategic benefits to the **Logistics Company**:

- **Elimination of Redundancy and Human Error:** Normalizing relational data up to BCNF ensures that every piece of transactional data is stored in exactly one place. This eliminates data discrepancies, prevents conflicting status updates, and maintains clean data across all departments.
- **Unprecedented Operational Visibility:** By utilizing MongoDB Atlas to store tracking logs, customer support agents and external clients can retrieve real-time shipment updates in milliseconds. This significantly reduces customer service call volumes and increases customer satisfaction.
- **Unmatched System Scalability:** The hybrid design ensures that high-volume tracking updates do not lock up transactional SQL tables. The database can scale to handle millions of tracking updates without degrading the performance of order processing or inventory control systems.
- **Data-Driven Decision Making:** Management gains access to highly accurate, real-time operational reports. Accurate warehouse stock levels prevent stock-outs, while precise courier tracking enables optimization of transit routes and fairer performance evaluations.

1.5 Methodology and Tools



To ensure a structured, high-quality development lifecycle, the project team adopted an **Agile Database Development Methodology**. This approach emphasizes iterative design, continuous integration, and rigorous testing across distinct phases, allowing the team to align database structures with physical logistics workflows.

The development lifecycle progressed through several key stages:

Phase 1: Problem Identification

Requirements Gathering and Conceptualization: Analyzed 3PL blueprints to define essential entities, attributes, and operational milestones, identifying key fragmentation bottlenecks in the current workflow.

Phase 2: Analysis & Documentation

Iterative Entity-Relationship Modeling: Mapped conceptual connections using collaborative tools to establish cardinalities, optionalities, and primary key structures, ensuring all business rules for the **Logistics Management System** were documented.

Phase 3: Database Design

Relational Normalization: Applied formal rules to transition the SQL model from 1NF to BCNF, eliminating data anomalies and ensuring structural efficiency.

Dual-Engine Configuration: Split the architecture into hybrid components: structured relational tables for MySQL and semi-structured JSON schemas for MongoDB Atlas.

Phase 4: Implementation

Data Synchronization and ID Mapping: Implemented a mapping strategy to maintain consistent unique identifiers across both engines, preventing data silos and ensuring cross-platform integrity.

Collaborative Integration and Testing: Merged scripts and seed data via GitHub using a multiple commits workflow and peer review to verify system performance and reliability.

1.5.1 Software and Hardware Tools

The implementation was supported by a robust stack of industry-standard tools:

- **Database Engines:** * **MySQL Server 8.0:** Used as the relational database engine, running locally for secure, ACID-compliant transaction processing and structured inventory management.
 - **MongoDB Atlas:** Used as the cloud-hosted NoSQL document store, providing scalable, highly available storage for dynamic shipment tracking collections.
- **Design and Development Interfaces:**
 - **MySQL Workbench 8.0:** Used for writing SQL DDL/DML scripts, executing structural queries, and generating physical relational diagrams.
 - **MongoDB Compass:** Used as the graphical user interface to interact with cloud collections, analyze document schemas, and test aggregation pipelines.
- **Collaborative and Development Platforms:**

- **Git & GitHub:** Used for collaborative code integration, revision tracking, and peer reviews, maintaining a clear development history.
- **Visual Studio Code (VS Code):** Used as the primary code editor for drafting JSON document payloads, and markdown documentation.

Chapter 2: Database Design

2.1 System Overview

The Logistics Management System is designed to manage key operations such as customer records, order processing, shipment tracking, inventory control, and warehouse management. It uses both MySQL for structured data and MongoDB for flexible data storage.

The system includes main entities such as Customer, Order, Product, Shipment, Courier, Warehouse, and Inventory. Customers place orders, orders contain products, and each order generates a shipment. Shipments are assigned to couriers and handled through warehouses, while inventory tracks product quantities.

Overall, the system improves data organization, reduces redundancy, and allows efficient tracking of logistics activities.

2.2 Identify Entities and Relationships

The foundation of the database design lies in identifying the core entities that drive the physical and administrative operations of the **Logistics Company**. Through detailed analysis of the organization's workflows, six primary entities were identified:

2.2.1 Customer

The Customer entity represents any individual or corporate entity that contracts the **Logistics Company** to ship goods. Key attributes include a unique customer identifier, personal and corporate names, contact phone numbers, email addresses, and primary billing or physical locations.

The Customer entity acts as the primary driver of the system, initiating the fulfillment pipeline by placing orders. It maintains a **One-to-Many (1:N)** relationship with the Order entity, as a single customer can place many orders over time, but each order is placed by exactly one customer.

2.2.2 Order

The Order entity represents a specific business transaction placed by a customer. Attributes include a unique order identifier, the date and time of purchase, the total financial amount, and the current processing state (e.g., Pending, Paid, Cancelled, Fulfilled).

The Order entity acts as the transactional hub of the system. It maintains a **Many-to-Many (M:N)** relationship with the Product entity, as a single order can contain multiple products,

and a single product can be included in many different orders. This relationship is resolved using a junction table named `Order_Items`.

2.2.3 Product

The Product entity represents the catalog of items handled by the logistics network. Attributes include a unique product identifier, product name, standardized Stock Keeping Unit (SKU), weight, volume, and categorization.

In addition to its relationship with orders, the Product entity is linked to physical storage locations. It maintains a **Many-to-Many (M:N)** relationship with the Warehouse entity, as a single warehouse can store many different products, and a single product can be distributed across multiple warehouses. This relationship is resolved using a junction table named `Warehouse_Inventory`.

2.2.4 Warehouse

The Warehouse entity represents the physical storage facilities operated by the **Logistics Company**. Attributes include a unique warehouse identifier, facility name, geographical location, and maximum storage capacity.

The Warehouse entity is directly linked to inventory management and shipment routing. It maintains a **One-to-Many (1:N)** relationship with the Shipment entity, as a single warehouse can serve as the origin for many shipments, but each shipment originates from a single, specific warehouse.

2.2.5 Courier

The Courier entity represents the delivery drivers and transport personnel responsible for moving goods from warehouses to final destinations. Attributes include a unique courier identifier, full name, phone number, vehicle type, and driver's license number.

The Courier entity is critical for tracking last-mile transit performance. It maintains a **One-to-Many (1:N)** relationship with the Shipment entity, as a single courier can be assigned to deliver many shipments over time, but each shipment is assigned to exactly one courier during its delivery phase.

2.2.6 Shipment

The Shipment entity represents the physical fulfillment process of an order in transit. Attributes include a unique shipment identifier, the associated order ID, origin warehouse ID, assigned courier ID, tracking number, ship date, and delivery status (e.g., In Transit, Out for Delivery, Delivered).

The Shipment entity is unique because it serves as the core link in both database systems. While it maintains strict relational foreign key links to Orders, Couriers, and Warehouses in

the MySQL database, its dynamic tracking data is synchronized as a document collection in MongoDB Atlas to enable high-velocity tracking updates.

2.3 Conceptual & Logical Database Design

2.3.1 Entity-Relationship Diagram (ERD)

The conceptual Entity-Relationship Diagram (ERD) establishes the logical structure of our database, detailing the exact cardinalities, optionalities, and referential pathways that govern our tables. This diagram serves as our authoritative architectural blueprint, ensuring that all physical tables implemented in MySQL match the business logic of the **Logistics Management System**.

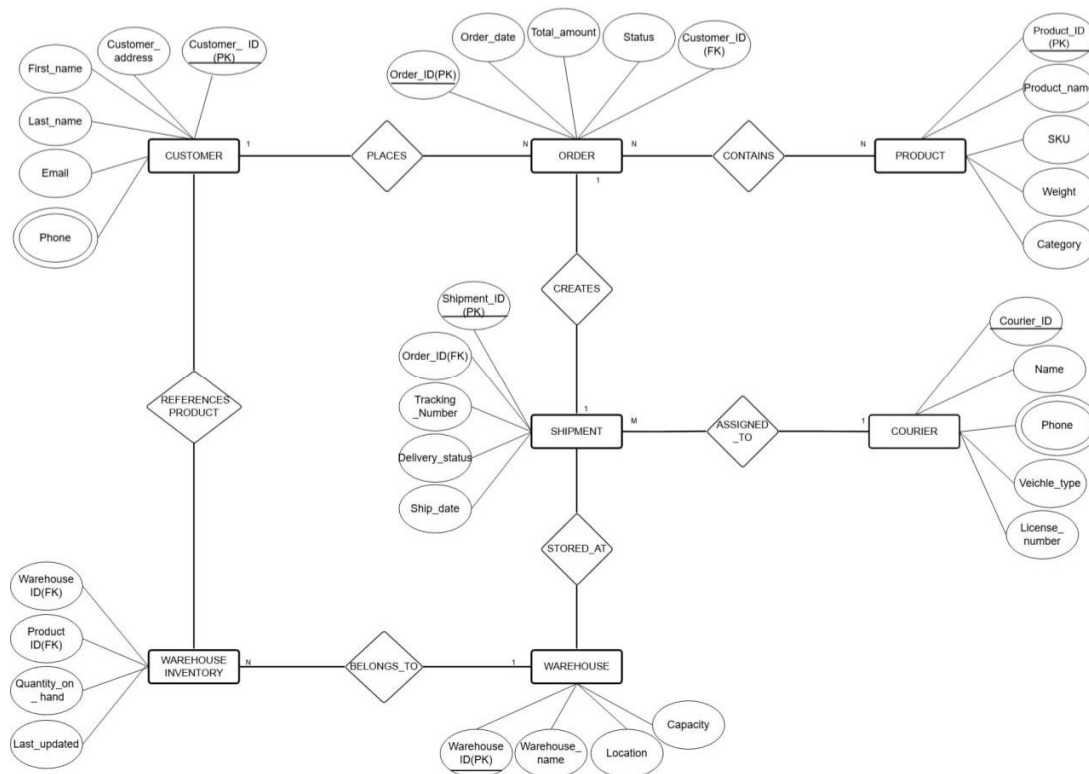
Management System.

Main entities in the system include:

- Customer
- Order
- Product
- Order_Item
- Shipment
- Courier
- Warehouse
- Warehouse_Inventory

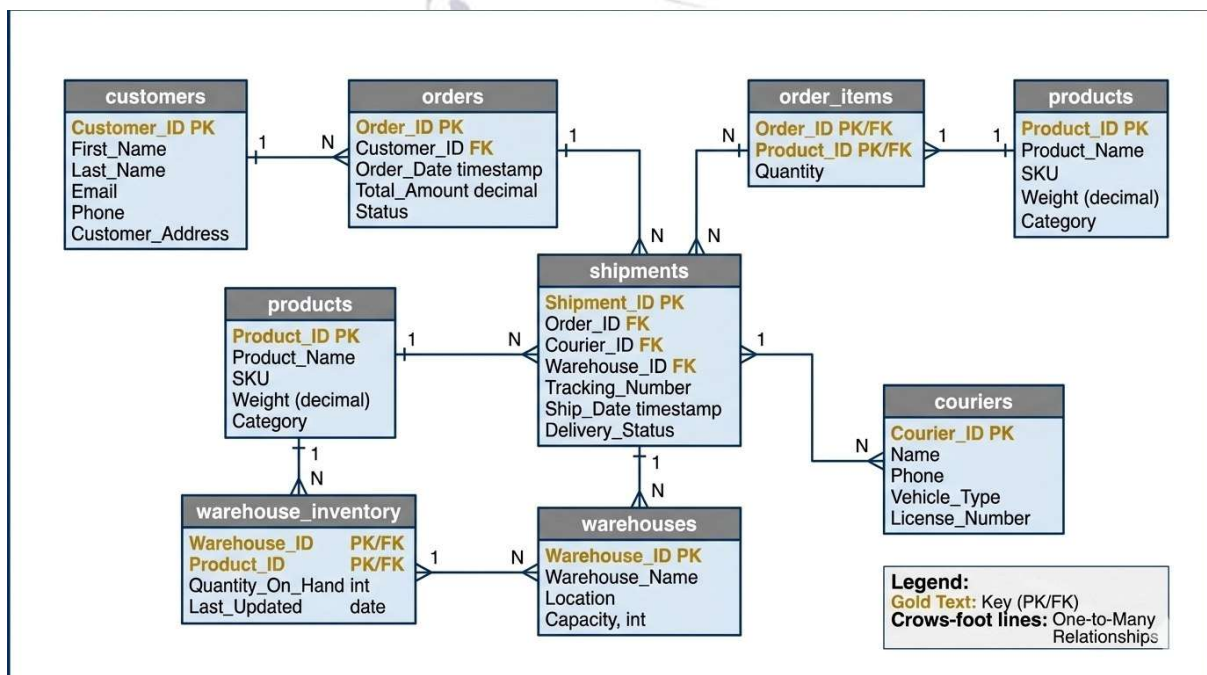
Relationship description:

- A **Customer** *places* one or more **Orders**, while each order belongs to only one customer (One-to-Many relationship).
- An **Order** *contains* multiple **Products**, and a product can appear in multiple orders. This creates a Many-to-Many relationship, which is resolved using the **Order_Item** entity.
- Each **Order** *generates* one **Shipment**, and each shipment is linked to a single order (One-to-One relationship).
- A **Shipment** is *handled by* one **Courier**, while a courier can handle multiple shipments (One-to-Many relationship).
- A **Shipment** is *processed through* one **Warehouse**, while a warehouse can manage multiple shipments (One-to-Many relationship).
- A **Warehouse** stores multiple **Products**, and each product can be stored in multiple warehouses. This Many-to-Many relationship is resolved using the **Warehouse_Inventory** entity.



2.3.2 Logical Schema Translation

To translate the conceptual ERD into a physical database, the entities and relationships are mapped into a structured relational schema. In this schema, primary keys are underlined, and foreign keys are designated with lines pointing to their parent table:



2.3 Normalization up to BCNF

To guarantee database integrity, prevent data anomalies, and eliminate redundant storage, the database schema underwent systematic normalization up to **Boyce-Codd Normal Form (BCNF)**. This rigorous process involved analyzing every functional dependency within our entities to ensure that only clean, logical structures were implemented.

2.3.1 First Normal Form (1NF)

A relation is in 1NF if and only if all attribute domains contain only atomic, indivisible values, and there are no repeating groups.

In our initial draft, the Customers table stored multiple phone numbers and physical delivery addresses in single, comma-separated fields. To achieve 1NF, we split these values. Phone numbers were restricted to a single primary contact field, and the Customer_Address was broken down into atomic components: street address, city, and region. In the Order_Items table, rather than storing list arrays of products purchased inside a single order record, we separated each item into its own distinct row, establishing a clean, tabular structure.

2.3.2 Second Normal Form (2NF)

A relation is in 2NF if it is in 1NF and every non-prime attribute is fully functionally dependent on the entire primary key, thereby eliminating partial dependencies. This rule is particularly important for junction tables with composite primary keys.

For example, in our composite table Warehouse_Inventory, the primary key is a combination of (Warehouse_ID, Product_ID). In an unnormalized state, if we stored the physical address of the warehouse in this table, that address would depend only on Warehouse_ID and not on Product_ID. This partial dependency would violate 2NF. To resolve this, we removed all warehouse details from the inventory table, leaving only the foreign key Warehouse_ID and placing the warehouse details securely inside the dedicated Warehouses table.

2.3.3 Third Normal Form (3NF)

A relation is in 3NF if it is in 2NF and there are no transitive dependencies, meaning no non-prime attribute determines any other non-prime attribute. Every non-key attribute must depend directly on the primary key, and only the primary key.

In our draft Orders table, we included the attributes Customer_ID, Customer_Name, and Customer_Email. This created a transitive dependency:

$$\text{Order_ID} \rightarrow \text{Customer_ID} \rightarrow \text{Customer_Name}$$

Because customer contact details depend on the Customer_ID and not directly on the Order_ID, this structure violated 3NF, creating severe update anomalies (e.g., updating a customer's email would require updating every order they ever placed). To achieve 3NF, we stripped all customer details from the Orders table, leaving only the foreign key Customer_ID to reference the Customers table.

2.3.4 Boyce-Codd Normal Form (BCNF)

A relation is in BCNF if and only if for every one of its non-trivial functional dependencies:

$$X \rightarrow Y$$

X is a superkey. BCNF is a stricter version of 3NF, resolving anomalies that can occur when tables contain multiple overlapping candidate keys. Let us analyze our normalized Warehouse_Inventory table. The composite primary key is (Warehouse_ID, Product_ID).

The functional dependency is:

$$\{\text{Warehouse_ID, Product_ID}\} \rightarrow \text{Quantity_On_Hand}$$

Since the combination of Warehouse_ID and Product_ID is a candidate key (and the primary key), the left side of our dependency is a superkey. There are no other functional dependencies in this table; attributes like warehouse capacity or product SKU codes are housed in their respective parent tables where their determining attributes are the primary keys. Thus, every table in our schema satisfies BCNF. This mathematical guarantee ensures that our database is free from insertion, update, and deletion anomalies.

2.3.5. Before Normalization (Unnormalized Form - UNF)

In the unnormalized state, data is stored in large, flat files. This leads to **multivalued attributes** (multiple phone numbers in one cell) and **redundant data** (repeating warehouse locations for every item in stock).

Table: Inventory_Customer_Flat

Customer_ID	Name	Phone_Numbers (Multivalued)	Warehouse_ID	Location	Product	Qty
0001	Abebe	+251934672210, +251911456789	4900	Addis Ababa	Laptop	50
0001	Abebe	+251934672210, +251911456789	4901	Nazret	Laptop	20
0002	Sara	+251980432999	4900	Addis Ababa	Mouse	100

Issues:

1NF Violation: The Phone_Numbers cell for Abebe contains two values, making it non-atomic.

3NF Violation: The Location depends on the Warehouse_ID, not the primary key of the transaction, causing "Update Anomalies" (if a warehouse moves, you must update every row).

2.5.6. After Normalization (BCNF Optimized)

By applying the normalization rules, we decompose the flat file into specialized tables. Each piece of information is stored in exactly one place.

1. Table: Customer_Phones (Resolves 1NF) Each phone number is now an individual record, ensuring atomicity.

Phone_ID	Customer_ID	Phone_Number	Phone_Type
1	0001	+251934672210	Mobile
2	0001	+251911456789	Work
3	0002	+251980432999	Mobile

2. Table: Warehouses (Resolves 3NF) Warehouse details are extracted to prevent location redundancy.

Warehouse_ID	Warehouse_Name	Location
4900	Central Hub	Addis Ababa
4901	East Annex	Nazret

3. Table: Warehouse_Inventory (BCNF Compliance) Inventory levels are determined strictly by the composite superkey (Warehouse_ID, Product_ID).

Warehouse_ID	Product_ID	Quantity_On_Hand	Last_Updated
4900	801	50	2026-05-01
4901	801	20	2026-05-02
4900	882	100	2026-05-01

2.4 Design Inputs, Outputs, and Reports

To translate this database architecture into actionable business intelligence for the **Logistics Company**, the system is designed to process specific operational inputs and generate clean, structured outputs and reports.

2.4.1 System Inputs (Forms and Data Entry)

Input forms are designed to allow users to enter and manage data efficiently within the system. The following forms are included:

- **Customer Form**
Used to capture customer details such as first name, last name, email, phone number, and address.
- **Order Form**
Allows entry of new orders, including customer selection, order date, status, and total amount.
- **Product Form**
Used to add and manage product details such as product name, SKU, weight, and category.
- **Order Item Form**
Enables users to select products for a specific order and specify quantities.
- **Shipment Form**
Captures shipment details including order reference, courier, warehouse, tracking number, and shipment status.
- **Courier Form**
Used to store courier information such as name, phone number, vehicle type, and license number.
- **Warehouse Form**
Allows entry of warehouse details including name, location, and capacity.
- **Inventory Form**
Used to update and monitor product quantities in each warehouse.

Example Customer Form:

Field Name	Data Type	Input Type	Description
Customer_ID	Integer	Auto-generated	Unique identifier
First_Name	Varchar	Text Field	Customer first name
Last_Name	Varchar	Text Field	Customer last name
Email	Varchar	Email Field	Customer email
Phone	Varchar	Text Field	Contact number
Address	Text	Text Area	Customer address

2.4.2 System Outputs (Receipts and Tracking Visuals)

Outputs represent the processed data displayed to users for decision-making:

- **Customer List:** displays all registered customers and their contact details.
- **Order Summary:** Shows details of orders including customer, date, total amount, and status.
- **Shipment Tracking View:** Displays shipment status, tracking number, and delivery progress.
- **Inventory Status:** shows available product quantities in each warehouse.

Example order Summary Output:

Order_ID	Customer_Name	Order_Date	Total_Amount	Status
550	Abebe Kebede	2026-04-30	250 ETB	Delivered

2.4.3 Operational Reports (Business Intelligence)

The system is optimized to generate comprehensive reports to guide business decisions:

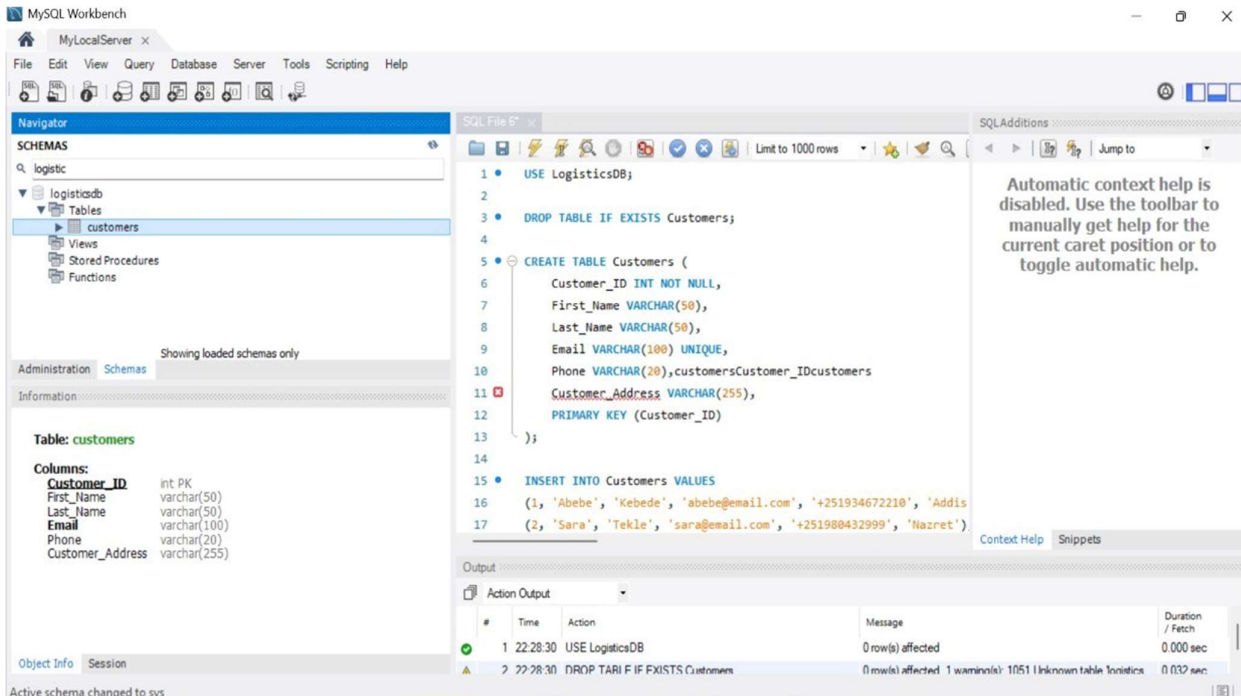
- **Daily Orders Report:** Lists all orders placed within a specific day.
- **Shipment Report:** Shows shipment details, including delivered and pending shipments.
- **Inventory Report:** Provides information on stock levels across warehouses.
- **Customer Activity Report:** Displays customer order history and activity.
- **Courier Performance Report:** Summarizes deliveries handled by each courier and their statuses.
- **Inventory Status and Stock Level Alerts:** A weekly report identifying products whose quantities are below defined thresholds, helping warehouse managers plan timely restocks.
- **Courier Performance and Delivery Speed Analysis:** Aggregates shipment records to calculate the average delivery time for each courier, helping dispatchers evaluate performance and identify delivery bottlenecks.
- **Regional Volume and Warehouse Throughput:** Summarizes shipment volumes across different cities and regions, helping management allocate physical resources, trucks, and staffing to high-demand areas.

Example Daily Orders Report:

Date	Total Orders	Total Revenue
2026-05-02	2	1000 ETB

Chapter 3: Implementation

3.1 MySQL Implementation



The relational database was implemented using MySQL. Based on the normalized schema, tables were created with primary and foreign keys to ensure data integrity and proper relationships.

Each entity from the ER diagram was converted into a relational table.

Example implementation:

Customer table stores customer details

Shipment table manages shipment records and links customers

Vehicle table stores vehicle information

Driver table stores driver details

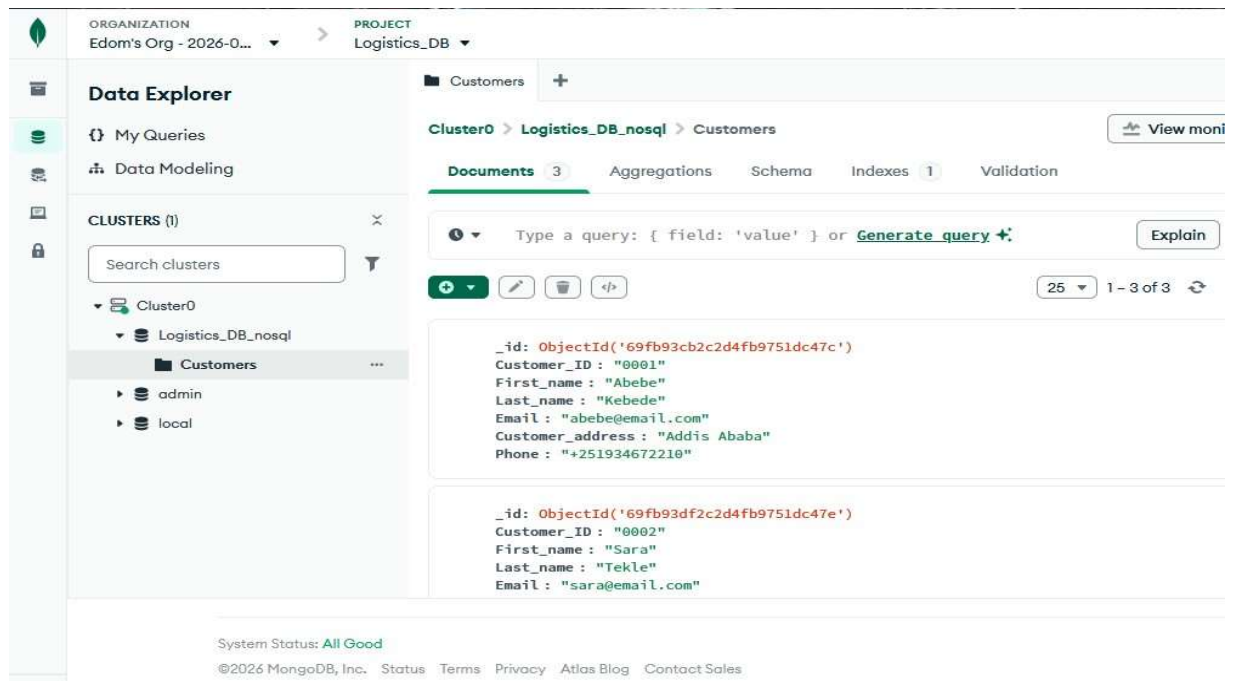
Relationships were enforced using foreign key constraints.

MySQL was used to handle structured data and ensure consistency across related tables.

3.2 MongoDB Implementation

In addition to the relational model, MongoDB was used to implement a NoSQL version of the logistics system. The data was stored in collections instead of tables.

Each collection represents an entity such as: Customers and Shipments



3.3 Sample Data Insertion

To validate the operational logic of the Logistics Management System, a comprehensive dataset was populated across both database environments. This data was designed to simulate real-world scenarios, including multiple contact points for customers and varied shipment statuses.

3.3.1 MySQL Relational Data

The relational data focus was on maintaining referential integrity across the normalized tables. Note that the Phone attribute is handled through the customer_phones table to comply with BCNF standards.

Customer and Contact Records:

SQL

-- Inserting into the primary Customer entity

```
INSERT INTO customers (Customer_ID, First_Name, Last_Name, Email,
Customer_Address)
```

VALUES

```
(1, 'Abebe', 'Kebede', 'abebe@email.com', 'Addis Ababa'),
```

```
(2, 'Sara', 'Tekle', 'sara@email.com', 'Nazret');
```

-- Resolving the multivalued Phone attribute (BCNF Compliance)

```
INSERT INTO customer_phones (Customer_ID, Phone_Number, Phone_Type)
VALUES
```

```
(1, '+251934672210', 'Mobile'),
```

```
(1, '+251911456789', 'Work'),
```

```
(2, '+251980432999', 'Mobile');
```

Logistics and Warehouse Records:

SQL

```
INSERT INTO warehouses (Warehouse_ID, Warehouse_Name, Location, Capacity)
```

```
VALUES (3200, 'Central Hub', 'Addis Ababa', 5000);
```

```
INSERT INTO shipments (Shipment_ID, Order_ID, Courier_ID, Warehouse_ID,
Tracking_Number, Ship_Date, Delivery_Status)
```

```
VALUES (9001, 550, 6023, 3200, 'ETH-99A-22', '2026-04-30', 'Delivered');
```

3.3.2 MongoDB Document Data

Data in MongoDB was inserted as rich JSON documents. This allows for the flexible storage of shipment history and nested customer details, optimized for high-speed retrieval in the user interface.

Shipment Collection Entry:

JavaScript

```
db.shipments.insertOne({
  "Shipment_ID": "9001",
  "Order_ID": "550",
  "Tracking_Number": "ETH-99A-22",
  "Delivery_status": "Delivered",
  "Ship_date": "2026-04-30",
  "Courier_ID": 6023,
  "Warehouse_ID": 3200
});
```

3.4 Challenges, Improvements, and Testing

3.4.1 Technical Challenges

During the implementation of the logistics database system, several architectural challenges were navigated:

- **Normalization Complexity:** Achieving BCNF in MySQL was initially difficult, particularly when managing many-to-many relationships between Products and Warehouses.
- **Hybrid Consistency:** Maintaining "Single Source of Truth" integrity between the relational (MySQL) and NoSQL (MongoDB) models required a strict synchronization strategy to avoid data drift.
- **NoSQL Schema Design:** Selecting between **Embedding** (storing items inside an order) and **Referencing** (using IDs) in MongoDB was a significant hurdle. We eventually chose embedding for Order History to reduce the need for application-level joins.

3.4.2 System Improvements

The following refinements were made during the Agile development cycles:

- **Schema Refactoring:** Resolved multivalued attributes (like Phone Numbers) by decomposing them into separate entities, reaching full **BCNF compliance**.
- **Constraint Enforcement:** Implemented strict FOREIGN KEY constraints in MySQL to automate referential integrity.
- **Performance Optimization:** Strategic indexing was applied to MongoDB Tracking_Number fields, significantly reducing search latency for customer-facing queries.

3.4.3 Debugging and Error Resolution

Category	Identified Bug	Resolution
Integrity	Parent-Child Violation: Errors occurred when inserting Orders before the corresponding Customer record existed.	Enforced sequential insertion logic and added ON DELETE CASCADE where appropriate.
NoSQL	Duplicate Document Entries: MongoDB allowed duplicate delivery_id entries because a unique index was missing.	Created a Unique Index on the delivery_id field within the MongoDB collection.
Synchronization	Data Drift: Shipment statuses in MySQL did not always match the cached status in MongoDB.	Revised the update logic to ensure a "Dual-Write" operation triggers for status changes.

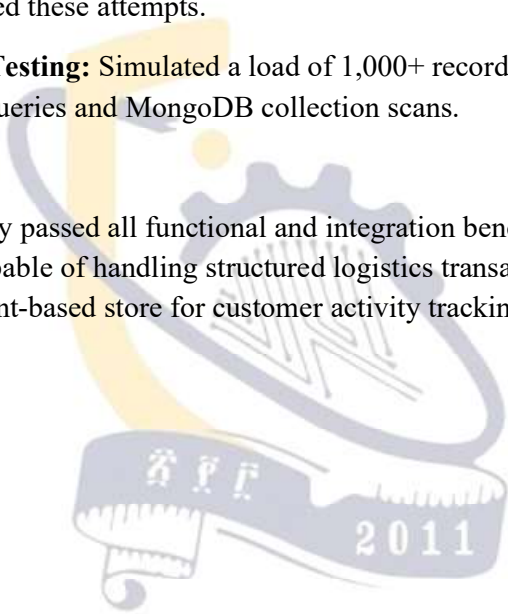
3.4.4 Testing Protocols

To verify the robustness of the system, four distinct testing phases were executed:

1. **Unit Testing:** Individual CRUD operations (Create, Read, Update, Delete) were tested on each of the 10 MySQL tables to ensure data was being stored according to the defined types.
2. **Integration Testing:** We verified that data successfully flowed from a MySQL Order to a MongoDB Shipment Record without loss of information.
3. **Data Integrity Testing:** Specifically targeted the **BCNF** structure by attempting to insert invalid data (e.g., a shipment to a non-existent warehouse). The system correctly blocked these attempts.
4. **Performance Testing:** Simulated a load of 1,000+ records to evaluate the efficiency of SQL JOIN queries and MongoDB collection scans.

3.4.5 Final Result

The system successfully passed all functional and integration benchmarks. The hybrid architecture proved capable of handling structured logistics transactions while providing the flexibility of a document-based store for customer activity tracking.



A large, faint watermark logo is centered on the page. It features a circular emblem with a gear-like border. Inside the circle, there is a stylized 'A' and the year '2011' at the bottom. Below the emblem, the word 'AASTU' is written in a large, bold, sans-serif font.

AASTU

Annexes

Annex A: Raw Data Samples

A.1 MongoDB (NoSQL) – Shipment Records (JSON Format)

JSON

```
[  
  {  
    "Shipment_ID": "9001",  
    "Order_ID": "550",  
    "Tracking_Number": "ETH-99A-22",  
    "Delivery_status": "Delivered",  
    "Ship_date": "2026-04-30",  
    "Courier_ID": "6023",  
    "Warehouse_ID": "3200"  
  },  
  {  
    "Shipment_ID": "9002",  
    "Order_ID": "551",  
    "Tracking_Number": "ETH-11B-33",  
    "Delivery_status": "In Transit",  
    "Ship_date": "2026-05-02",  
    "Courier_ID": "6001",  
    "Warehouse_ID": "3045"  
  },  
  {  
    "Shipment_ID": "9003",  
    "Order_ID": "552",  
    "Tracking_Number": "ETH-44C-55",  
    "Delivery_status": "Pending",  
    "Ship_date": "2026-05-02",
```

```
"Courier_ID": "6005",  
"Warehouse_ID": "3105"  
}  
]
```

A.2 MongoDB (NoSQL) – Customer Records (JSON Format)

JSON

```
[  
  {  
    "Customer_ID": "0001",  
    "First_name": "Abebe",  
    "Last_name": "Kebede",  
    "Email": "abebe@email.com",  
    "Customer_address": "Addis Ababa",  
    "Phone": ["+251934672210"]  
  },  
  {  
    "Customer_ID": "0002",  
    "First_name": "Sara",  
    "Last_name": "Tekle",  
    "Email": "sara@email.com",  
    "Customer_address": "Nazret",  
    "Phone": ["+251980432999"]  
  }  
]
```

A.3 SQL – Sample Insert Statements

SQL

-- Inserting Normalized Customer Data

```
INSERT INTO customers (Customer_ID, First_Name, Last_Name, Email,  
Customer_Address) VALUES
```



```
(1, 'Abebe', 'Kebede', 'abebe@email.com', 'Addis Ababa'),  
(2, 'Sara', 'Tekle', 'sara@email.com', 'Nazret');
```

-- Inserting Multivalued Phone Attributes

```
INSERT INTO customer_phones (Customer_ID, Phone_Number, Phone_Type) VALUES  
(1, '+251934672210', 'Mobile'),  
(2, '+251980432999', 'Mobile');
```

-- Inserting Shipment Data

```
INSERT INTO shipments (Shipment_ID, Order_ID, Courier_ID, Warehouse_ID,  
Tracking_Number, Ship_Date, Delivery_Status) VALUES  
(9001, 550, 6023, 3200, 'ETH-99A-22', '2026-04-30', 'Delivered'),  
(9002, 551, 6001, 3045, 'ETH-11B-33', '2026-05-02', 'In Transit');
```

Annex B: Code Snippets

B.1 MongoDB Insert Operations

JavaScript

```
db.shipments.insertMany([  
  {  
    Shipment_ID: "9001",  
    Order_ID: "550",  
    Tracking_Number: "ETH-99A-22",  
    Delivery_status: "Delivered",  
    Ship_date: "2026-04-30",  
    Courier_ID: "6023",  
    Warehouse_ID: "3200"  
  }  
]);
```

B.2 SQL Schema Creation (BCNF Normalized)

SQL

```
CREATE TABLE customers (  
    Customer_ID INT PRIMARY KEY,  
    First_Name VARCHAR(50),  
    Last_Name VARCHAR(50),  
    Email VARCHAR(100),  
    Customer_Address VARCHAR(255)  
);
```

```
CREATE TABLE customer_phones (  
    Phone_ID INT PRIMARY KEY AUTO_INCREMENT,  
    Customer_ID INT,  
    Phone_Number VARCHAR(20),  
    FOREIGN KEY (Customer_ID) REFERENCES customers(Customer_ID)  
);
```

```
CREATE TABLE shipments (  
    Shipment_ID INT PRIMARY KEY,  
    Order_ID INT,  
    Courier_ID INT,  
    Warehouse_ID INT,  
    Tracking_Number VARCHAR(100),  
    Ship_Date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    Delivery_Status VARCHAR(30),  
    FOREIGN KEY (Order_ID) REFERENCES orders(Order_ID),  
    FOREIGN KEY (Courier_ID) REFERENCES couriers(Courier_ID),  
    FOREIGN KEY (Warehouse_ID) REFERENCES warehouses(Warehouse_ID)  
);
```

B.3 Query Examples

- **SQL (Relational):**

SQL

```
SELECT s.Shipment_ID, s.Tracking_Number, s.Delivery_Status
```

```
FROM shipments s
```

```
WHERE s.Delivery_Status = 'Delivered';
```

- **MongoDB (NoSQL):**

JavaScript

```
db.shipments.find({ Delivery_status: "Delivered" });
```

Annex C: Diagrams

C.1 SQL ER Diagram (Relational Model)

Description: The Entity-Relationship (ER) diagram illustrates relationships between core entities such as Customers, Orders, Shipments, Products, and Warehouses. Primary and foreign keys define structured relationships ensuring referential integrity and BCNF compliance.

C.2 MongoDB Document Schema Illustration

Description: The MongoDB schema demonstrates a flexible document-based structure where shipment and customer data are stored as JSON documents. Relationships are handled through embedded fields or references rather than strict joins, optimized for horizontal scaling.

Annex D: Comparison Tables

Feature	SQL (Relational Database)	MongoDB (NoSQL Database)
Data Structure	Tables (rows and columns)	Documents (JSON/BSON)
Primary Keys	Required (e.g., Customer_ID)	Optional (automatic _id)
Normalization	Optimized for BCNF	Denormalized/Embedded
Scaling	Vertical (better hardware)	Horizontal (more servers)

Conclusion

The implementation of the Logistics Management System demonstrates the successful integration of relational and non-relational database technologies to solve complex supply chain challenges. By strictly adhering to BCNF normalization within MySQL, the system ensures high data integrity and the elimination of redundant anomalies, while the strategic use of MongoDB provides the flexibility and performance required for real-time customer activity tracking. Following an Agile development methodology allowed for iterative refinements, ensuring that the final schema is not only theoretically sound but also optimized for practical business reporting. Ultimately, this hybrid architecture provides a scalable, robust foundation capable of generating critical insights—from inventory alerts to courier performance—essential for data-driven decision-making in modern logistics.

References

- Addis Ababa Science and Technology University. (2026). *Database management systems (DBMS) course PowerPoints*. Department of Software Engineering.
- GitHub Repository (2026). [AASTU-DB-Project-Group-4-Section-B](#). Includes SQL scripts NoSQL document and ERD collections.
- MySQL, "MySQL 8.0 Reference Manual," Oracle Corporation, 2026. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>
- MongoDB, "The MongoDB Database Manual," MongoDB, Inc., 2026. [Online]. Available: <https://www.mongodb.com/docs/manual/>
- R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA: Pearson, 2017.
- IEEE, "IEEE Standard for Software Quality Assurance Processes," *IEEE Std 730-2014*, pp. 1-138, 2014.